

SPEC-001-GEDCOM Normalization & Enrichment Platform

Background

Organizations and power users need a robust way to ingest *any* GEDCOM file (legacy 5.5/5.5.1 through modern 7.x), normalize the data, enrich it (standardized dates/places/names, geocoding, source harmonization), and publish it to multiple backends (relational, document, and graph—Neo4j) for downstream research, deduplication, and sharing. Existing tools often fail on edge cases (ANSEL encoding, CONT/CONC folding, custom tags, malformed cross-refs, huge media sections). This project establishes a standards-compliant, test-driven pipeline and API that reliably parses, validates, transforms, and exports genealogical data at scale.

Requirements

Observed stack & scope (from your repo) - Language: **Python 3.13** (packaged project under `GEDCOM_Parser_Project/src/gedcom_parser`), rich test suite in `tests/`. - Current modules: `loader/` (tokenizer, tree_builder, value_reconstructor), `entities/` (registry, extraction), `normalization/` (names, dates, places, events), `postprocess/` (place plan, entity resolution, scoring), `core/` (context, pipeline), `exporter/` (JSON), `events/`, `occupation/`, `utils/`, `logger`, plus generated entity/attachment models and tag schemas under `schema/gedcom55`. - CLI entry points: `run.py`, `parse_gedcom.py`, `main.py` with `Pipeline` orchestration.

Must-have (M)

1. **Standards-complete parsing for GEDCOM 5.5/5.5.1** (you're close): robust level/xref model; **CONT/CONC** joining; pointers (INDI/FAM/SOUR/OBJE/NOTE) integrity; strict/lenient modes with rule codes.
2. **Encoding & I/O hardening**: detect **ANSEL**, ASCII, Latin-1, UTF-8 (with/without BOM); fallback transliteration where ANSEL glyphs lack Unicode; stream parsing for very large files.
3. **Validation engine** with rule catalog (per tag + context) and severity (error/warn/info), auto-fixes where safe (trim invalid dates, collapse whitespace, repair line wraps, normalize `@U` unlinked pointers).
4. **Normalization**: EDTF mapping for dates; place canonicalization with jurisdiction tokens + gazetteer lookup; name parsing supporting multi-cultural patterns; SOUR/CITATION structuring; media linking & checksums.
5. **Entity Registry** guarantees: bidirectional INDIVIDUAL↔FAMILY (FAMC/FAMS), unique xref enforcement, orphan detection, and repair suggestions.
6. **Exports**: structured JSON (present), **round-trip GEDCOM writer** (5.5.1), **Neo4j CSV** emitters (nodes/edges), **Postgres** relational dump (SQL schema + COPY/CSV).
7. **Observability & QA**: structured logs, per-entity audit trail, metrics (parse rate, rules hit), reproducible fixtures incl. malformed edge cases.
8. **Privacy controls**: living-person filters, redaction policies, configurable output scopes.
9. **Performance**: memory-bounded streaming tokenizer, chunked registry materialization, O(n) single pass where possible; optional multiprocessing for enrichment.

Should-have (S)

1. GEDCOM **7.x (read)** with compatibility layer; **GEDZIP (read)** for embedded media.
2. Pluggable **gazetteers** (GeoNames, WOF, Nominatim) with local cache.
3. **Deduplication**: extend existing `postprocess/entity_resolution.py` with probabilistic linkage, merge scripts, and provenance.
4. **Rules DSL** for custom org transforms and tag aliases.

Could-have (C) 1. Web UI for job submission and validation triage. 2. GEDCOM-X export; JSON-LD context for linked-data export.

Won't-have (W) for MVP 1. Media OCR, face recognition, or external tree reconciliation.

Method

Architecture (high level)

```
@startuml
skinparam shadowing false
skinparam rectangle {
  BorderColor #888
  RoundCorner 10
}

rectangle CLI/API {
  [main.py] --> (Pipeline)
}

rectangle Pipeline {
  (Tokenizer) --> (Tree Builder)
  (Tree Builder) --> (Value Reconstructor)
  (Value Reconstructor) --> (Entity Registry)
  (Entity Registry) --> (Normalization)
  (Normalization) --> (Postprocess: Linkage/Dedupe)
  (Postprocess: Linkage/Dedupe) --> (Exporters)
}

rectangle Parsers {
  (Tokenizer) -down-> (Encoding Detector)
}

rectangle Exporters {
  (JSON)
  (GEDCOM Writer)
  (Neo4j CSV)
  (Postgres/Parquet)
}

@enduml
```

Parsing & Validation

- **Tokenizer:** keep your `loader/tokenizer.py`; add: BOM stripping, encoding probe (chardet-like heuristic + `HEAD.CHAR`), **ANSEL decoder** with fallback map, hard limits (max line length), and streaming iterators.
- **Tree Builder:** current `loader/tree_builder.py` is good—add: strict level transitions checker, pointer uniqueness map, cross-ref graph to detect dangling/duplicate xrefs.
- **Value Reconstruction:** confirmed `loader/value_reconstructor.py` handles **CONT/CONC**; add table-driven rules (tag→joiner rules) and whitespace policy.
- **Validation:** introduce `validator/` package with rules like `V-HEAD-CHAR-001`, `V-INDI-NAME-002`, etc., attach to nodes; emit JSON report + summary metrics.

Canonical Data Model (in-memory)

- Keep `entities/` & `generated_entities.py`; ensure each entity has: `id` (xref), `uuid`, `type`, `names`, `events`, `attrs`, `links`, `sources`, `media`, `notes`, `confidences`, `provenance`.
- Use **EDTF** normalized dates; places as token arrays with geocodes; names as structured blocks (prefix, given, surname, suffix, aka, script).

Relational Schema (Postgres)

```
@startuml
entity INDIVIDUAL {
    id PK // xref @I1@
    uuid UNIQUE
    sex
    primary_name_id FK
    birth_event_id FK
    death_event_id FK
    living_flag
}
entity NAME {
    id PK
    individual_id FK
    given
    surname
    prefix
    suffix
    aka
    script
}
entity FAMILY {
    id PK
    husband_id FK
    wife_id FK
}
}
```

```

entity FAMILY_CHILD {
    family_id FK
    child_id FK
    order_no
}
entity EVENT {
    id PK
    owner_type // INDIVIDUAL/FAMILY
    owner_id FK
    type // BIRT, MARR, DEAT, ...
    date_raw
    date_edtf
    place_id FK
    details_json
}
entity PLACE {
    id PK
    name
    parts_json // ["Hamlet","Parish","County","Country"]
    lat
    lon
    source
}
entity SOURCE { id PK; title; author; pub_info; repo_id FK }
entity CITATION { id PK; source_id FK; owner_type; owner_id; page; text;
quality }
entity MEDIA { id PK; hash; mime; path; title; note }
entity NOTE { id PK; text }

INDIVIDUAL ||--o{ NAME
INDIVIDUAL ||--o{ EVENT
FAMILY ||--o{ EVENT
FAMILY ||--o{ FAMILY_CHILD
PLACE ||--o{ EVENT
SOURCE ||--o{ CITATION
@enduml

```

Neo4j Graph Model

- **Nodes:** `Person{uuid,sex,living}`, `Family{uuid}`, `Event{type,edtf}`, `Place{lat,lon,name}`, `Source{title}`.
- **Rel:** `(:Person)-[:SPOUSE_OF]-(:Person)`, `(:Person)-[:CHILD_OF]->(:Person)`, `(:Person)-[:MEMBER_OF]->(:Family)`, `(:Event)-[:OCCURRED_AT]->(:Place)`, `(:Person)-[:PARTICIPATED_IN]->(:Event)`, `(:Event)-[:CITED_BY]->(:Source)`.
- Export with idempotent CSVs + `MERGE` CYPHER loader; include provenance attributes and confidence.

Enrichment

- **Dates → EDTF:** expand `normalization/date_normalization.py` with full GEDCOM grammar (ranges, about, from/to, before/after, dual-dating) mapped to EDTF.
- **Places:** tokenization + jurisdiction planner (you already have `postprocess/place_plan_applier.py`); add gazetteer adapters + cache.
- **Names:** keep `normalization/name_normalization.py`; add script detection, transliteration, and nickname dictionaries.

Deduplication/Record Linkage

- Extend `postprocess/entity_resolution.py` with:
- Blocking keys (phonetic surname + YOB ± 1 + region).
- Similarity: Jaro-Winkler for names, date distance, place distance (haversine), household context boost.
- Merge policy: conservative, conflict matrix, full provenance retention.

Exporters

- Keep `exporter/json_exporter.py`.
- Add `exporter/gedcom_writer.py` (round-trip fidelity)
- Add `exporter/neo4j_csv.py` and `exporter/postgres_dump.py`.

Implementation

1. **Repository hygiene**
2. Move virtualenv out of repo; enforce `src/` layout, `pyproject.toml`, Ruff + MyPy + pytest.
3. Split `generated_*` into `schema/` with regeneration script.
4. **Parsing hardening**
5. Add `io/encoding.py` (BOM/encoding/ANSEL), `io/streams.py` (chunked read), line guardrails.
6. Cross-ref index & validator with O(1) lookups; strict/lenient modes.
7. **Validator service**
8. `validator/` with rule registry; emit SARIF/JSON; CLI `validate` command.
9. **Normalization**
10. Complete EDTF; place adapters (GeoNames/WOF/Nominatim) with on-disk cache; nickname & surname variants.
11. **Persistence & Export**
12. Implement Postgres DDL + COPY; Neo4j CSV emit + loader cypher; GEDCOM writer.
13. **Privacy & QA**
14. Living-filter module; redaction policy; gold-set fixtures; property-based tests.
15. **Performance**
16. Streaming parse; lazy entity construction; batch exporters; optional multiprocessing for enrichment.

Milestones

1. **M1 – Parser & Validator:** encoding/ANSEL, strict tree, rule engine, 95% test pass on supplied corpora.

2. **M2 – Normalization Core:** EDTF dates, place/name normalization, SOUR/NOTE fidelity.
3. **M3 – Exports:** JSON stable schema, GEDCOM writer, Neo4j CSV + loader, Postgres dump.
4. **M4 – Deduplication v1:** blocking + similarity + conservative merge.
5. **M5 – Scale & Privacy:** streaming for 100MB+, living filter, audit trail.
6. **M6 – GEDCOM 7.x & GEDZIP (read).**

Gathering Results

- **Validation KPIs:** % files parsed, errors per 1k lines, rule hit histogram.
- **Fidelity:** round-trip delta (parser→writer→parser) < 0.5% semantic loss.
- **Enrichment quality:** geocode hit rate, EDTF coverage, dedupe precision/recall on labeled pairs.
- **Performance:** lines/sec, peak RSS, export throughput.

Need Professional Help in Developing Your Architecture?

Please contact me at sammuti.com :)